ECE 650 – Assignment 1

Test Report: Alternative Implementation

File tested: other-code-test/ece650-a1.py

Section	What it covers
1. Summary	Overall result and the list of problems, by severity
2. How it was tested	The six groups of tests and what each checks
3. What works	Behaviour that passed every test
4. Problems and fixes	Each problem with evidence, why it matters, and the code change
5. Results after the fixes	Every test re-run on the fixed version
6. Open questions	Points to confirm on Piazza before submitting
Appendix	The complete set of changes as a diff

1. Summary

The program finds the correct graph for the assignment's sample, never crashed on any input, and keeps error messages off standard output. Its geometry is sound: it uses exact fractions, handles overlapping streets, and runs quickly.

Five problems were found. The most serious is the number format: every coordinate is printed with two decimals, e.g. `(2.00,2.00)` where the assignment's sample shows `(2,2)`. Because marking compares output exactly, this alone could fail every graph test case.

#	Problem	Severity	Fix size
1	Whole-number coordinates printed with decimals: (2.00,2.00) instead of (2,2)	**High**	3 lines
2	Vertex numbers change between gg commands	**Medium**	10 lines
3	Accepts malformed lines that should be errors (missing spaces, digits or symbols in names, + signs, names differing only by spacing)	**Medium**	10 lines
4	Rejects a valid street that returns to an earlier point	**Low**	3 lines
5	When a street touches or overlaps itself, an edge can skip over a vertex	**Low**	15 lines

Outcome: with all five fixes applied (about 40 changed lines, listed in the appendix), the program passes every test in this report, including 1,000 random inputs compared against an independent implementation. The fixed copy is saved as `other-code-test/ece650-a1-fixed.py`; the original file is unchanged.

2. How it was tested

All tests are in `other-code-test/run_tests.py`. The script only runs the programs; it never edits them. It runs the original by default, or another file if one is named:

```
python3 run_tests.py                      # the original
python3 run_tests.py ece650-a1-fixed.py   # the fixed copy
```

Group	What it does	What it checks
1. Sample	Runs the input from the assignment PDF.	Correct graph after add, mod and rm, and whether the text matches the PDF exactly.
2. Error cases	32 single-line cases, each run after a valid street: bad spacing, bad names, bad coordinates, wrong commands, duplicate streets.	Is each bad line rejected with an Error message, and is each valid line accepted?
3. Output streams	Mixes bad lines with a valid gg.	Errors appear only on standard error, never in the graph output.

Group	What it does	What it checks
4. Numbering	Runs gg, adds unrelated streets, runs gg again.	Existing vertices keep their numbers.
5. Random comparison	400 random inputs of 2–7 streets with 2–5 points each, sometimes removing a street. A further 600 on a small grid to force overlaps.	The graph matches the one from the ../a1 implementation, compared by coordinates so numbering differences don't count.
6. Speed	60 random streets, about 6,000 vertices.	Finishes in reasonable time.

The random comparison (group 5) is a technique called **differential testing**: two programs written independently should give the same answer, so any difference points to a bug in one of them. Each difference found was then traced by hand to decide which program was wrong.

3. What works

- **Sample graphs are correct.** 10 vertices and 9 edges, then 5 and 4 after mod, then empty after rm, matching the assignment.
- **No crashes.** Exit code 0 on every one of over 1,000 inputs. A catch-all `except Exception` in the main loop means even an unexpected bug prints an error rather than stopping the program.
- **Errors are separated.** Every error starts with `Error:` and goes to standard error.
- **Exact geometry.** Intersections use `Fraction`, so the same crossing found from two segment pairs is recognised as one vertex. Overlapping streets are handled.
- **Most error cases are handled well**, with clear messages: duplicate add, mod or rm of unknown streets, one point only, missing quotes, empty names, missing or malformed brackets, decimal coordinates, extra text after gg or rm, unknown commands, and names that differ only in case.
- **Fast.** About 6,000 vertices in 0.36 seconds.

4. Problems and fixes

Problem 1 – Coordinates printed with decimals

Severity: **High**

WHAT HAPPENS

Every coordinate is printed with two decimal places, even whole numbers:

```
Expected (assignment PDF)      Printed by this program
  1: (2,2)                       1: (2.00,2.00)
  2: (4,2)                       2: (4.00,2.00)
```

WHY IT MATTERS

The assignment says output must match *perfectly* because marking is automated. If the marker compares text, every test case with a graph would fail, which is most of the marks.

CAUSE

```
def fmt(v):
    # only round when printing; TAs asked for python's built-in round()
    s = f"{round(float(v), 2):.2f}"
    return "0.00" if s == "-0.00" else s
```

The format `:.2f` always prints two decimals.

FIX

Print whole numbers as they are and only use two decimals for fractions:

```
def fmt(v):
    # whole numbers print as-is, like the sample: (2,2)
    if v.denominator == 1:
        return str(v.numerator)
    # only round when printing; TAs asked for python's built-in round()
    s = f"{round(float(v), 2):.2f}"
    return "0.00" if s == "-0.00" else s
```

> **Check first:** the code comment mentions a TA instruction. If a Piazza post says *all* coordinates must have two decimals, keep the original and ignore this fix. If it only says how to round non-whole numbers, apply it.

Problem 2 – Vertex numbers change between gg commands

Severity: **Medium**

WHAT HAPPENS

Numbers are handed out fresh on every gg, in order of coordinates. Adding a street that doesn't touch the existing ones still renumbers everything:

```
add "A" (0,0) (4,4)
add "B" (0,4) (4,0)
gg                      -> (0,0) is vertex 1
add "C" (-5,-5) (-5,5)
add "D" (-6,0) (-4,0)
gg                      -> (0,0) is now vertex 6
```

In the test, all five original vertices changed number.

WHY IT MATTERS

The spec allows any vertex names, so this may not lose marks on its own. But the PDF sample keeps numbers stable (vertex 6 is still 6 after mod), and later assignments feed this output into a vertex-cover solver, where shifting numbers make results across commands hard to compare. The code comment “keeps the numbering stable between runs” is only true when the graph is identical.

FIX

Store the number table on the `Streets` object so it survives between gg calls, and only give numbers to new points:

```
class Streets:
    def __init__(self):
        self.data = {}
        self.ids = {}       # point -> vertex number, kept between gg calls
        self.next_id = 1
```

and at the end of `build_graph`, replace the renumbering with:

```
    # new points get the next free number; old points keep theirs
    ids = streets.ids
    for p in sorted(points):
        if p not in ids:
            ids[p] = streets.next_id
            streets.next_id += 1
    vertices = {ids[p]: p for p in points}
```

Problem 3 – Malformed lines are accepted

Severity: **Medium**

WHAT HAPPENS

Input	Original result	Should be
`add"B" (0,4) (4,0)`	accepted	error: no space after command
`add "B"(0,4) (4,0)`	accepted	error: no space after name
`add "B" (0,4)(4,0)`	accepted	error: no space between points
`add "B1" (0,4) (4,0)`	accepted	error: digit in name
`add "B#!" (0,4) (4,0)`	accepted	error: symbols in name
`add "B" (+1,4) (4,0)`	accepted	error: + sign
`add "A " ...` when `"A"` exists	accepted as a new street	error: same street
`"B Street"` then `"B Street"`	two different streets	error: same street

WHY IT MATTERS

The assignment gives 25% of the marks for detecting errors, and the hidden tests use “honest mistakes” a user makes. Missing spaces and stray characters are exactly that kind of mistake. A comment in the code says skipping the space is deliberate; if Piazza allows it, keep that part as is. Names that differ only in spaces should still be the same street, since the assignment says the amount of whitespace doesn't matter.

FIX

Require whitespace between parts, allow only `-` as a sign and only ASCII digits, and check name characters:

```
_POINT = r"\(\s*(-?[0-9]+)\s*,\s*(-?[0-9]+)\s*\)"
point_re = re.compile(_POINT)
# every point needs at least one space in front of it
point_list_re = re.compile(r"(?:\s+" + _POINT + r")+\s*")

# a space is required between the command and the name
name_re = re.compile(r'\s+"([^"\n]*)"(.*)', re.DOTALL)
cmd_re = re.compile(r"\s*([A-Za-z]+)(\s.*|)", re.DOTALL)
valid_name_re = re.compile(r"[A-Za-z ]+")
```

In `parse_line`, change `cmd_re.match(line)` to `cmd_re.fullmatch(line)` so the command must be followed by a space or the end of the line. In `read_name`, check the characters and collapse spaces before returning:

```
    if not valid_name_re.fullmatch(name):
        raise ParseError("street name can only contain letters and spaces")
    # collapse extra spaces so "Weber  St " and "Weber St" are the same street
    return " ".join(name.split()), m.group(2)
```

Changing `\d` to `[0-9]` also matters: in Python, `\d` matches digits from any language, such as ², which `int()` can't convert.

Problem 4 – A street that returns to an earlier point is rejected

Severity: **Low**

WHAT HAPPENS

```
add "B" (0,0) (1,1) (0,0)
Error: the same point shows up twice in this street
```

WHY IT MATTERS

A street that loops back, such as a cul-de-sac or a ring road, is valid input. Only the *same point twice in a row* is a problem, because it makes a segment of length zero.

FIX

```
    for a, b in zip(pts, pts[1:]):
        if a == b:
            raise ParseError("the same point appears twice in a row")
```

This replaces `if len(set(pts)) != len(pts)`, which rejected any repeat.

Problem 5 – Edges can skip over a vertex

Severity: **Low**

WHAT HAPPENS

In 35 of the 400 random inputs the graph differed from the reference. All 35 involved a street that touches or overlaps itself. Two small examples:

```
add "A" (0,0) (4,4) (4,0) (2,2)
add "B" (3,0) (3,3)
gg
Original adds edge (0,0)-(3,3), but (2,2) is a vertex lying between them.

add "A" (0,0) (6,6) (6,0) (2,4)
add "B" (2,2) (4,4)
gg
Original adds edge (2,2)-(4,4), but (3,3) is a vertex lying between them.
```

WHY IT MATTERS

The rules say two vertices are joined only if one can be reached from the other *without passing another vertex*. These edges break that rule. The inputs are unusual, so the marker may never test them.

CAUSE

For each segment, the original lines up only that segment's own crossings plus its two ends. A vertex created somewhere else that also lies on the segment is left out. That happens when the end of another segment of the same street sits on it (example 1), or when a crossing falls inside a stretch where two streets overlap (example 2).

FIX

Collect all vertices first, then for each segment use every vertex that lies on it. Add a helper:

```
def _on_segment(p, a, b):
    # p is on segment a-b if it's on the same line and inside the box
    cross = (b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0] - a[0])
    return cross == 0 and _in_box(p, a, b)
```

and in `build_graph`, build the full set of vertices before the edge loop, then use it:

```
    ends = set()
    for (_, start, end), on_seg in zip(segs, hits):
        if on_seg:
            ends.add((Fraction(start[0]), Fraction(start[1])))
            ends.add((Fraction(end[0]), Fraction(end[1])))
    points = crossings | ends

    edge_pairs = set()
    for (_, start, end), on_seg in zip(segs, hits):
        ...
        on_line = [p for p in points if _on_segment(p, start, end)]
        ...
        chain = sorted(on_line, key=...)
```

TRADE-OFF

Checking every vertex against every segment is slower. The 6,000-vertex test went from 0.36 s to about 3.4 s, roughly the same as the ../a1 program, which does the same check. For

assignment-sized inputs (tens of vertices) the difference is not noticeable.

A cheaper version that only added other segments' ends was also tried. It fixed example 1 but not example 2, and still disagreed on 4 of 400 random inputs, so the full check is recommended.

5. Results after the fixes

Test	Original	Fixed
Sample graph correct	Pass	Pass
Sample text matches the PDF exactly	Fail (2.00,2.00)	Pass (2,2)
Malformed lines rejected (8 cases in Problem 3)	0 of 8	8 of 8
Street returning to an earlier point accepted	No	Yes
Other error cases (23)	Pass	Pass
Errors only on standard error	Pass	Pass
Vertex numbers kept across gg	5 of 5 changed	None changed
400 random inputs vs ../a1	35 mismatches	0 mismatches
600 dense random inputs (overlaps, loops)	not run	0 mismatches
Crashes across all inputs	0	0
60 streets, 6,000 vertices	0.36 s	3.4 s

Agreement with ../a1 doesn't prove both are right, since two programs could share a mistake. But the two were written differently, and each disagreement found earlier was checked by hand against the assignment's rules.

6. Open questions to confirm on Piazza

Question	Why it matters
Should whole-number coordinates be printed as (2,2) or (2.00,2.00)?	Decides whether Problem 1 is a bug. The original code cites a TA instruction about round().
Is a space required between the command, the name and each point?	The original deliberately allows add"B" with no space. If that's allowed, skip the spacing part of Problem 3.
Which characters may a street name contain?	The fix allows letters and spaces only, matching every example in the assignment.
Must vertex numbers stay the same between gg commands?	If not, Problem 2 is optional.

Appendix – Complete diff

The full set of changes between `ece650-a1.py` (original, lines starting with `-`) and `ece650-a1-fixed.py` (lines starting with +). Also saved as `other-code-test/fixes.diff`.

```
@@ -14,13 +14,15 @@
 # ---------- parsing ----------

 # an integer point like (2,-1); spaces are fine inside the parens
-_POINT = r"\(\s*([+-]?\d+)\s*,\s*([+-]?\d+)\s*\)"
+_POINT = r"\(\s*(-?[0-9]+)\s*,\s*(-?[0-9]+)\s*\)"
 point_re = re.compile(_POINT)
-point_list_re = re.compile(r"(?:\s*" + _POINT + r")+\s*")
+# every point needs at least one space in front of it
+point_list_re = re.compile(r"(?:\s+" + _POINT + r")+\s*")

-# leading space before the quote is optional, so add"Weber St" still works
-name_re = re.compile(r'\s*"([^"\n]*)"(.*)', re.DOTALL)
-cmd_re = re.compile(r"\s*([A-Za-z]+)(.*)", re.DOTALL)
+# a space is required between the command and the name
+name_re = re.compile(r'\s+"([^"\n]*)"(.*)', re.DOTALL)
+cmd_re = re.compile(r"\s*([A-Za-z]+)(\s.*|)", re.DOTALL)
+valid_name_re = re.compile(r"[A-Za-z ]+")


 def parse_line(line):
@@ -28,7 +30,7 @@
     if not line.strip():
         return None

-    m = cmd_re.match(line)
+    m = cmd_re.fullmatch(line)
     if not m:
         raise ParseError("unknown command (expected add, mod, rm or gg)")
     cmd, rest = m.groups()
@@ -58,7 +60,10 @@
     name = m.group(1)
     if not name.strip():
         raise ParseError("street name is empty")
-    return name, m.group(2)
+    if not valid_name_re.fullmatch(name):
+        raise ParseError("street name can only contain letters and spaces")
+    # collapse extra spaces so "Weber  St " and "Weber St" are the same street
+    return " ".join(name.split()), m.group(2)


 def read_points(text):
@@ -73,8 +78,9 @@
     pts = [(int(x), int(y)) for x, y in point_re.findall(text)]
     if len(pts) < 2:
         raise ParseError("a street needs at least two points")
-    if len(set(pts)) != len(pts):
-        raise ParseError("the same point shows up twice in this street")
+    for a, b in zip(pts, pts[1:]):
+        if a == b:
+            raise ParseError("the same point appears twice in a row")
     return pts


@@ -85,6 +91,8 @@

     def __init__(self):
```

```
         self.data = {}
+        self.ids = {}       # point -> vertex number, kept between gg calls
+        self.next_id = 1

     def add(self, name, pts):
         key = name.lower()
@@ -123,6 +131,12 @@

 def _in_box(p, a, b):
     return _between(p[0], a[0], b[0]) and _between(p[1], a[1], b[1])
+
+
+def _on_segment(p, a, b):
+    # p is on segment a-b if it's on the same line and inside the box
+    cross = (b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0] - a[0])
+    return cross == 0 and _in_box(p, a, b)


 def intersect(p1, p2, p3, p4):
@@ -181,31 +195,44 @@
                 hits[i].add(p)
                 hits[j].add(p)

-    points = set(crossings)
-    edge_pairs = set()
+    # segment ends that become vertices (because their segment has a crossing)
+    ends = set()
+    for (_, start, end), on_seg in zip(segs, hits):
+        if on_seg:
+            ends.add((Fraction(start[0]), Fraction(start[1])))
+            ends.add((Fraction(end[0]), Fraction(end[1])))
+    points = crossings | ends

+    edge_pairs = set()
     for (_, start, end), on_seg in zip(segs, hits):
         if not on_seg:
             continue  # nothing crosses it, so this segment adds nothing

         start = (Fraction(start[0]), Fraction(start[1]))
         end = (Fraction(end[0]), Fraction(end[1]))
-        points.add(start)
-        points.add(end)

+        # every vertex lying on this segment, not just this segment's own
+        # crossings -- otherwise an edge could jump over a vertex that came
+        # from somewhere else (a street touching itself, or an overlap)
+        on_line = [p for p in points if _on_segment(p, start, end)]
+
         # line the points up from start to end (projection onto the
         # direction vector -- no sqrt needed since we only sort by it)
         dx, dy = end[0] - start[0], end[1] - start[1]
-        chain = sorted(on_seg | {start, end},
+        chain = sorted(on_line,
                        key=lambda p: (p[0] - start[0]) * dx + (p[1] - start[1]) * dy)

         for p, q in zip(chain, chain[1:]):
             if p in crossings or q in crossings:
                 edge_pairs.add(frozenset((p, q)))

-    # sorting the points first keeps the numbering stable between runs
-    ids = {p: n for n, p in enumerate(sorted(points), start=1)}
-    vertices = {n: p for p, n in ids.items()}
+    # new points get the next free number; old points keep theirs
+    ids = streets.ids
```

```
+    for p in sorted(points):
+        if p not in ids:
+            ids[p] = streets.next_id
+            streets.next_id += 1
+    vertices = {ids[p]: p for p in points}
     edges = {tuple(sorted(ids[p] for p in pair)) for pair in edge_pairs}
     return vertices, edges

@@ -213,6 +240,9 @@
 # ---------- output ----------

 def fmt(v):
+    # whole numbers print as-is, like the sample: (2,2)
+    if v.denominator == 1:
+        return str(v.numerator)
     # only round when printing; TAs asked for python's built-in round()
     s = f"{round(float(v), 2):.2f}"
     return "0.00" if s == "-0.00" else s
```